CS3070 – Exploration 6 -- Digital Objects, Directories, Sharing & Capabilities
Summer 2025

SCORE: 82.5/90

Questions

1. Review the resources on files, pipes, devices, and streams.

1.5 a. What is a stream? A stream is a string, or sequence of bits/bytes that flows between a process and some Input/Output device or file.

2.0 b. In what way is a stream a unifying principle for files, pipes, and devices? Stream is a unifying principle because files contain a stream, input/output devices generate/absorb streams, and a pipe communicates a stream. A stream is the conduit of data.

2.5 c. How does the stream concept benefit the user kernel API? The shell? The stream concept benefits the user kernel API because it doesn't need to unify different object types under a single interface. It is simplified to generic terms with create, delete, open, close, read, write, etc. This simplifies the API and allows portability.

For the shell, everything looks like a stream. This uniform view enables redirection, pipelines, and composition. With redirection (> file, < file), the shell binds a process's standard input or output stream to a file stream. With pipelines, the shell connects the output stream of one process directly to input stream of another. Because every program reads and writes streams, the shell can compose commands freely. Any two commands can be chained into the same stream, regardless of what they do internally.

3.0 2. The basic file system (Level 6) manages all the files for all the users of a computer system. Symbolic object names, directories, and directory trees are not part of a basic file system; they are implemented at the next higher level (Level 7).

A file is stored on the disk as a tree of records listed in an index table. When a file is created, the file system gives the owner a pointer to the index table. Level 6 only deals with files by their pointers, not their symbolic names.

3.0 a. The basic file functions are **CREATE**, **DELETE**, **OPEN**, **CLOSE**, **READ**, **WRITE**. Review what these functions do with your team; note that **OPEN** only applies to files, while **READ/WRITE** apply only to buffers. Attest that you understand these functions.

We attest that we understand these functions.

b. What does the **OPEN** command do? Why not read and write to the index tree on the disk rather than the linear file image in a memory buffer?

OPEN(object) establishes an efficient and fast communication pathway between a process and the file.

3.0 The open command retrieves the file capability from the process's capability list, validates that the tag is a file and that the open bit is set. It then creates a memory buffer containing a linear copy of that file. All subsequent READ and WRITE operations go to this buffer and not directly to the disk. We do not read or write directly to the disk's index tree because disk input/output is slow and scattered around the disk. This creates a severe performance problem where the solution is using a buffer in RAM which makes access much faster and allows the kernel to enforce type and permission checks efficiently.

c. Suppose **A** is the pointer to a file (generated by the **CREATE** command) and **B** is the pointer to the buffer (generated by the **OPEN** command). Why are **A** and **B** different?

3.0 A and B are different because they refer to different objects. A is the file handle created by CREATE that points to the permanent file on disk. B is the buffer handle created by OPEN that points to a temporary buffer in memory holding the file's contents. The OS distinguished them to ensure performance and safety.

d. Is **OPEN(A)** a valid command? Explain?

3.0 Assuming CL[A] exists, OPEN(A) is a valid command. The OPEN command takes a position of a file capability in the process's capability list. The kernel then validates that entry (tag = file, open bit set) and returns a new buffer handle B. From then on, only B may be used for READ, WRITE, and CLOSE, while A is only valid for CREATE, DELETE, or further OPEN calls.

e. Is **READ(B)** a valid command? Explain?

3.0 READ(B) is a valid command. B is a buffer handle created by open. READ and WRITE operate only on buffer handles, not directly on file handles. The kernel enforces this by checking the capability's type field and ensuring that buffer objects can be used for CLOSE, READ, and WRITE.

3. Directories are tables associating symbolic names with the system pointers for the named objects. Thus a user who wants to create a file with symbolic name **S** must ask the basic file system (Level 6) for a new file with pointer **A**, and then ask the directory system to place **(S,A)** in a directory.

a. Why separate the directory functions from the basic file functions?

3.0 This is a deliberate OS level design choice to separate two distinct operations in regard to object management. This allows flexibility of naming, as directories are just mappings of symbolic names to pointers. If naming were dependent on the file system itself, every name change would require the entire directory to be modified. In addition, when we modify a file, we only

need to update the pointer in the directory (to the new file in memory) instead of having to change the whole table entry.

b. Outline a method to insert **(S,A)** into a directory (Level 7) without requiring the basic file system (Level 6) to call upward with a request "insert **(S,A)** into directory"?

3.0

Implement ATTACH (d, (S, A)) entirely in level 7. First, level 7 validates , S, and the public capability A. Next, OPEN the directory's backing file system (Level 6) and read its table. Then in level 7 insert (S, A) into the table in memory. Finally, it uses level 6 operations to write the updated table and close the directory's backing file to persist the updated table to disk.

c. A directory entry can point to any digital object created at Levels 7 and 6 in the user kernel. Explain how this makes directory trees possible. Is it possible for a request "insert **(S,D)** into a directory" to create loop back to a directory higher up in the tree?

3.0

At Level 6, the OS can create files which are just data objects stored in blocks of memory. At Level 7, the directory system creates tables that are name-pointer pairs as described above (S, A). At Level 7, the directory can either point to a Level 6 object or another Level 7 directory. This hierarchy allows us to create a tree structure with a directory being the root node.

No, it is not possible to insert (S,D) into a directory to create a loopback to a higher directory because this would violate the tree structure requirement. The ATTACH operation must validate that the target directory D is not an ancestor of the current directory by tracing the parent pointers upward. If D appears in this ancestral path, the insertion is rejected to prevent cycles that would make path resolution impossible and break the fundamental tree invariant of the directory system.

d. What is the benefit of storing the pointer to itself as **.** (dot) as the first entry in every directory? And of storing the pointer to its parent as **..** (double-dot) as the second entry?

3.0

This makes path operations (navigation and manipulation) relatively simple, since we do not need to specify the entire path name. For example, if we are in a directory and we need to read a certain file (i.e. 'nano file1.txt'), we can just enter ./file1.txt instead of having to enter the entire path name. This works in a similar manner when accessing objects in the parent directory.

e. For what is the **CD** (current directory) field in a virtual machine (Level 8) used?

3.0

The CD field in a VM is used to store the working directory of the user in session. This allows the user to make queries and access files within the current working directory without having to specify the entire path name.

f. For what is the **PATH** field in a virtual machine used?

3.0

The PATH field in a VM points to the set of directories which contain executable files when a user types a command. For instance, when entering "cat" the shell/VM consults the PATH field in order, checking each directory for a matching file that has the execute permission.

g. Every digital object is subject to sharing rules declared by the object's owner. This can be represented by associating an access control list with every object. Thus, a directory entry would be **(S,A,ACL)** , where **ACL** is the access control list. The **ACL** lists all users who have permission to access the file. The directory system will return an object pointer only if a selected directory entry's **ACL** explicitly gives the user permission. What is the benefit of an **ACL** for representing access permissions? Why is **ACL** not feasible as an exclusive access control mechanism in a large system?

1.5

An ACL is not feasible as an exclusive mechanism in large systems due to storage explosion and management complexity. Each object must store potentially thousands of user IDs, creating massive storage overhead that scales as O(users × objects). More critically, user management becomes intractable - adding or removing a single user requires updating ACLs across potentially millions of objects system-wide. This creates both performance bottlenecks and consistency problems that make ACLs impractical as the sole access control mechanism.

4. Suppose I post a readable file in my directories at **/myname/file**. The URL is **nps.edu/myname/file**. What is to prevent me from later substituting a new file, say with malware, at the same URL? All those linking to me did so in the belief my file contains no malware; but I fooled them. In effect the URL has been reused for a different file. Can you think of methods to keep this from happening with URLs?

3.0

One way we can prevent this is to have a trusted third party verify that the URL in question is what it portrays itself to be and not malicious in nature. Once this has been done, it can then create a hash of the webpage that is kept publicly available for anyone to reference. This way, when a user is attempting to access the webpage, they can cross reference their computed hash with the one that is made publicly available.

In addition to the idea of a hash, we can have the original author of the webpage digitally sign the webpage with their private key. If the author is a trusted user, public users can verify their digital signature before downloading the webpage.

5. Capabilities (managed at Level 5) are a method of generating unforgeable pointers to digital objects that authorize specific access and bypass the need for an ACL search. Discuss the following issues.

a. What could go wrong if users could modify capabilities?

3.0

If users could modify capabilities, the entire security model would collapse. Capabilities are the sole authorization mechanism - they are the permission, not just a reference to permission stored elsewhere. Any modification ability

allows users to forge arbitrary access rights, escalate privileges to any level, and access any object in the system. This fundamentally violates the unforgeability principle that makes capabilities secure, rendering the entire protection system worthless since there would be no trusted way to verify legitimate access rights.

b. Could a protected C-list implement the unforgeability ideal?

3.0 Yes, as C-lists are stored in protected kernel space and not accessible to any user process, which guarantees unforgeability once the capability is issued.

c. A distributed system is a network in which capabilities can be passed around and used from any machine. Why would a protected C-list not work in a distributed system?

3.0 Protected C-lists fail in distributed systems because protection is local to each machine's kernel space. When a capability containing a handle is sent to Machine B, that handle is meaningless in Machine B's kernel space and descriptor tables. Machine B has no way to resolve Machine A's internal handles to actual objects. The protection that makes handles unforgeable locally becomes a barrier to cross-machine object access, requiring a different mechanism like cryptographically signed public capabilities for network-wide validity.

d. Describe how cryptographic checksums would be used to guarantee unforgeability in a distributed system.

3.0 Public form of capabilities for network sharing includes the form (tag, access, host, handle, checksum). Any recipient of this public capability can verify it has not been modified since its creation by computing the hash of its (tag, access, host, handle) components and comparing the result with the checksum field. If this computed has matches the checksum field, the capability can be considered unaltered and therefore unforgeable in the distributed environment.

6. In the OS reference model [See *os99.pdf* and *The Art of OS online* Module 0.3] each level of the kernel software is the manager of a specific set of digital objects. At Levels 6 and higher, these managers return capabilities for the objects they create. They get their capabilities by downward calls to **CREATE_CAP** command at Level 5. (Level 5 also provides commands for **MAKE_PUBLIC** and **COPY**.) Capabilities are stored in user C-lists, which are part of their virtual machines. A user can refer to a capability indirectly, for example **CL[j]** is the capability at position **j** in the user's C-list. Users are not required to manage their capabilities by storing them in their address spaces.

a. When a file is created with the command **j = CREATE_FILE()**, the file system obtains from Level 5 a capability **c = CREATE_CAP(F,a,x)** , where **F** is the tag indicating *file*. **a** is the access code (set initially to all access), and **x** is the unique file number the file system has selected for the file. The file system stores **c** in an unused entry j of the user's C-list and returns **j** to the user.

Thereafter the user can call **OPEN(j)** to open the file and **CLOSE(j)** to close it. Capabilities in C-lists last until the owning process quits and the C-list is deleted. A user wanting to preserve a capability beyond the lifetime of that user's processes can copy capabilities into directories. Specify a kernel operation that does this.

3.0

To copy a capability into a directory for preservation beyond the lifetime of a process, a user would first use the MAKE_PUBLIC operation to transform an internal capability (from their C-list) into its public form, and then use the ATTACH operation provided by the directory manager. Specifically, j = COPY_CAP(i, mask) makes a copy of a capability in the C-list with potentially attenuated access rights, and then c = MAKE_PUBLIC(i) converts an internal capability (t,a,h) into a public capability (t,a,h,IP,check) that can be sent anywhere in the network. The ATTACH(d, (N,c)) operation then creates a new entry in directory d, associating a user-chosen symbolic name N with this public capability c.

b. Do capabilities stored in directories need cryptographic checksums?

3.0

Yes, as capabilities stored in directories are in their "public form". The public form of a capability includes a cryptographic checksum (check) to verify that it has not been modified since its creation and to ensure its unforgeability when shared outside the protection of its operating system.

c. Can users who want to see their capabilities in raw bit form do so?

0.0

No, capabilities are stored in a process's C-list, which is in kernel space and not accessible to the user.

d. A user process can perform a kernel function on an object by calling the kernel interface with the object capability as argument. How does the object manager verify that incoming capabilities point to objects it manages? For example, how does the file system verify that incoming parameter capabilities actually designate files?

3.0

Object managers verify that incoming capabilities designate objects they manage by checking the object type tag contained within the capability. This tag specifies the type of object, such as a file or process, and the object manager's generic operations read this field to route the request to the appropriate manager. For example, the file system verifies that a capability's tag is "file" to confirm it applies to a file.

e. The specification for **CREATE_CAP** says that the newly created capability **c** is placed at a position **j** in the C-list of the calling process. Suppose the process owner wants to make a copy of that capability in a directory (whose capability is) **d**. The obvious way to do this is to use the **MAKE_PUBLIC** command to attach a crypto checksum to **c**, yielding a new public capability

c', and then use the directory ATTACH command to place (N,c') in d. Show how to do this without violating the *downward call-upward return* rule.

3.0

1. User process (level 8) calls MAKE_PUBLIC(j) at Level 5, where j is the C-list position containing capability c.
2. Level 5 returns public capability c' to user process (level 8).
3. User process (level 8) calls ATTACH(d, (N, c')) at Level 7 Directory Manager.

f. The network protocol stack also uses levels. Data packets on the sending machine are passed downward through the levels, acquiring additional header bits from each level. On the receiving machine the packets move upward, being stripped of a level's special bits when they pass through. This is a level-structured dataflow rather than a call-return structured procedure flow as in an OS kernel. Why not use the same dataflow principle for the OS levels?

3.0

The OS kernel uses call-return structure to enforce protection boundaries and controlled access. Each level only exposes specific operations to higher levels, preventing unauthorized access to lower-level functions. Dataflow would allow data to pass through all levels without access control, violating the principle of least privilege where processes should only access the minimum necessary operations.

7. The mapping of capability to object uses two levels of indirection. The first level maps the handle to a descriptor, the second maps the descriptor to the specific memory holding the object. All the descriptors of an object manager can be stored in a large hash table controlled by that manager. The descriptor table can be quite large – for example, a typical file system can easily have ten million descriptors to manage.

a. Outline a method for mapping a capability handle to its specific descriptor in the hash table.

3.0

1. A hash function H is applied to the capability handle to reduce its bits to m-bit hash value r, which is an index in the table.
2. Each entry HT[r] is the start of a linked list of blocks, which has a (handle, descriptor) pair and pointer to next block.
3. The object manager searches the chain for this block that matches the specific handle to retrieve its descriptor.

b. Finding an entry in a large hash table can take several memory accesses, which means that mapping handle to descriptor can be slow. How can the mapping process be speeded up so that it is almost as fast as a single memory access?

3.0

Using a cache or "mapping accelerator" can speed up mapping processes, as after first references to the map can bypass the full linked list lookup and directly go to the cache.

c. Someone who dislikes all the mappings from capabilities to hash tables to file descriptor proposes a simplification: store the file descriptor in the directory entry and omit the hash tables. What makes this infeasible?

3.0

Location independence and sharing issues. For example, if a file descriptor were stored directly in every directory entry that referenced that object, then relocating the object would require finding and updating every single copy of its descriptor across all directories, which is wildly inefficient and cou ld require suspending all processes dependent on that file object. With handles and a descriptor table, only the mapping table needs updating when an object is moved, and all references remain valid.